

Exercícios

O objetivo desses exercícios é treinar a tradução lógica de como nós humanos pensamos, para algoritmos de máquina, nesse caso usando a linguagem Javascript.

Para cada exercício crie um arquivo chamado `exercicio-<número>.js` dentro de uma pasta no nosso projeto do github.

Os exercícios abaixo dão um problema de negócio e dicas de como estruturar ele em passos lógicos. Seu papel é criar o código Javascript que atende o que foi pedido

Exercício 1: A Memória da Máquina (Declaração de Variáveis)

O Problema de Negócio: Você está construindo a tela de checkout de um e-commerce. O sistema precisa calcular o valor final a ser cobrado do cliente. O carrinho tem um produto de R\$ 150,00, o frete custa R\$ 20,00 e o cliente tem um cupom de desconto de R\$ 30,00.

O Desafio: Declare as caixas (variáveis) para representar cada um desses valores isoladamente e, em seguida, crie uma última caixa que guarde o resultado da operação matemática entre eles.

Passo 1: Escreva os comentários (A Lógica)

```
// 1. Guardar o preço do produto
// 2. Guardar o valor do frete
// 3. Guardar o valor do desconto
// 4. Guardar o total (produto + frete - desconto)
// 5. Mostrar o total na tela
```

Exercício 2: O Inspetor de Estoque (Laços de Repetição)

O Problema de Negócio: Você recebeu uma remessa de 5 caixas de um produto. Cada caixa contém uma quantidade diferente de itens: `[10, 25, 5, 12, 8]`. O gerente precisa saber o total exato de itens que chegaram ao armazém. Lembre-se: o computador só consegue olhar para uma caixa por vez.

O Desafio: Use a técnica do "Papel em branco" (Acumulador) e um laço de repetição para somar o estoque.

Passo 1: Escreva os comentários (A Lógica)

```
// A lista de caixas que chegou
const caixas = [10, 25, 5, 12, 8];

// 1. Criar a variável acumuladora começando em 0 (o papel em branco)
// 2. Iniciar o laço para olhar caixa por caixa
// 3. Em cada volta, somar a quantidade da caixa atual com o valor que já
// 4. Fora do laço, mostrar o total do estoque
```

Exercício 3: A Catraca do Evento (Condições Booleanas)

O Problema de Negócio: Um sistema de catraca eletrônica precisa decidir se libera ou bloqueia a entrada de um funcionário. Para entrar, o funcionário precisa atender a **duas** regras simultaneamente: possuir o crachá ativo E ter nível de acesso maior ou igual a 3.

O Desafio: Crie as variáveis para representar o estado do funcionário e use a estrutura de decisão para imprimir "Acesso Liberado" ou "Acesso Negado". Experimente mudar os valores das variáveis para testar os dois caminhos.

Passo 1: Escreva os comentários (A Lógica)

```
// 1. Criar variável booleana informando se o crachá está ativo (true/fal
// 2. Criar variável numérica com o nível de acesso do funcionário
// 3. SE o crachá for verdadeiro E o nível de acesso for >= 3:
// 4.     Mostrar "Acesso Liberado"
// 5. CASO CONTRÁRIO:
// 6.     Mostrar "Acesso Negado"
```

Exercício 4: O Relatório Atrasado (Assincronicidade)

O Problema de Negócio: Você precisa exibir o nome de um cliente na tela. Porém, essa informação está guardada em um servidor lento. Se você tentar imprimir o nome imediatamente, o sistema vai mostrar dados vazios, porque a resposta ainda não chegou da internet.

O Desafio: Escreva uma função que saiba "pausar" a execução até que o banco de dados responda. (Nota: Para este exercício, usaremos uma função falsa `buscarClienteNoBanco()` que simula o tempo de rede).

Passo 1: Escreva os comentários (A Lógica)

```
// Função simulada que demora 2 segundos
function buscarClienteNoBanco() {
    return new Promise(resolve => setTimeout(() => resolve("Maria Silva"))
}

// 1. Criar uma função assíncrona para iniciar o sistema
// 2. Mostrar mensagem "Buscando dados..."
// 3. Pausar a execução e esperar o resultado da função buscarClienteNoBa
// 4. Salvar esse resultado numa variável
// 5. Mostrar a mensagem "Cliente encontrado:" junto com o nome
```

Exercício 5: O Desafio do Fechamento de Caixa (Integração)

Este é o teste final. Ele exige que o aluno estruture o raciocínio em camadas: o tempo, a repetição, a condição e a memória.

O Problema de Negócio: No fim do dia, o seu sistema precisa calcular o lucro líquido de uma loja. As transações do dia estão guardadas num servidor (demora para buscar). O servidor devolve uma lista de objetos. Cada objeto tem um `tipo` (que pode ser "venda" ou "despesa") e um `valor`.

O Desafio: Busque os dados no servidor. Leia transação por transação. Se for "venda", some o valor ao caixa. Se for "despesa", subtraia o valor do caixa. No final, mostre o saldo do dia.

```
// Função simulada da API (Não alterar)
function buscarTransacoesAPI() {
    return new Promise(resolve => setTimeout(() => resolve([
        { tipo: "venda", valor: 200 },
        { tipo: "despesa", valor: 50 },
        { tipo: "venda", valor: 100 },
        { tipo: "despesa", valor: 20 }
    ]), 1500));
}
```